

User Feedback Provides a Unique Signal that LLMs Can not Detect

Shachar Don-Yehiya^{1,2}

Leshem Choshen^{2,3,4}

Omri Abend¹

¹The Hebrew University of Jerusalem, ²IBM Research, ³MIT, ⁴MIT-IBM Watson AI Lab
{first.last}@mail.huji.ac.il

Abstract

Harnessing naturally occurring feedback from user interactions offers a promising learning signal for Large Language Models (LLMs). However, recent studies suggest this feedback is inherently noisy and difficult to leverage effectively. We challenge this conception by demonstrating that user feedback is a highly actionable signal for improvement, and that its perceived ineffectiveness stems from a systematic bias in current evaluation paradigms. To isolate the usefulness of feedback, we construct synthetic data with a definitive ground truth, alongside naturalistic data to validate that our findings hold in real-world scenarios. By comparing model revisions generated with and without access to feedback across both settings, we show that feedback-informed revisions resolve targeted issues at significantly higher rates than baseline revisions. Finally, we expose the root of the evaluation bias: when a model successfully fixes an issue exclusively due to feedback, LLM judges frequently fail to identify the genuinely corrected response, systematically preferring inferior baseline outputs instead.

1 Introduction

As Large Language Models (LLM) are being used in more open-ended tasks without clear success signals, aligning the models with human intent becomes challenging. Inspired by the spontaneous feedback humans provide in natural conversation (Vranjes et al., 2018; Bavelas and Gerwing, 2011; Bassiri, 2011; Werts et al., 1995; Pickering and Garrod, 2021), prior research has explored extracting naturally occurring feedback from user interactions, mainly with chat models (Don-Yehiya et al., 2025b; Lin et al., 2024; Shi et al., 2026). While naturally occurring feedback has high potential as a learning signal (Don-Yehiya et al., 2025a; Buening et al., 2026; Jin et al., 2025), recent work argues that it is inherently noisy and difficult to use effectively (Liu et al., 2025).

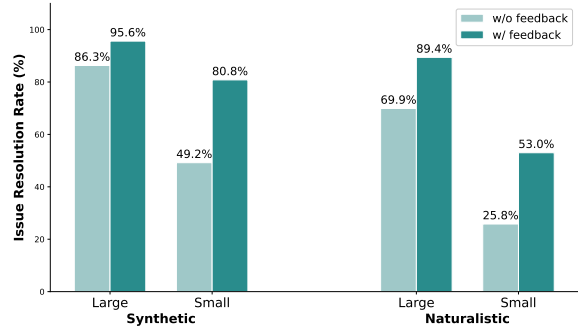


Figure 1: Issue resolution rates for large and small improvers on synthetic and naturalistic data. The synthetic data results represent the average across all corruption types. For both settings, providing feedback ("w/ feedback") yields a consistent positive delta over the baseline ("w/o feedback"). While baseline performance drops on the more challenging naturalistic data, the relative benefit of providing feedback remains strong.

This paper challenges this common conception. Through controlled experiments, we find that naturally occurring feedback is not inherently ineffective; on the contrary, it is a strong signal for improvement, but its contribution is masked by evaluation paradigms that are systematically biased against it.

To isolate the utility of feedback, we compare model revisions made with and without feedback on two types of data. We ensure feedback targets real issues and a ground truth is available through synthetic data, where we deliberately corrupt existing model responses (§2.1). In contrast, we validate our findings hold true in reality, by extracting user feedback from real conversations (§2.2).¹

Across both settings, we find that the feedback is highly effective (§5.1). With-feedback improvements resolve the targeted issues at significantly higher rates than their without-feedback

¹Code: <https://github.com/shachardon/feedback-blindspot> Data: <https://huggingface.co/datasets/shachardon/feedback-blindspot>

baselines—yielding a 9% – 32% increase in resolution on the synthetic data, and a replicated 16% – 27% increase on the real user data. This raises a critical question: if feedback demonstrably drives such improvements, why was it previously found to be unhelpful?

We investigate the evaluation mechanism as the primary source of confusion. We directly compare the two improvement variants using a strong LLM judge (Chiang and Lee, 2023; Liu et al., 2023). By isolating the instances across our data where the feedback-informed response successfully fixed the issue while the baseline response failed, we uncover a critical vulnerability in current evaluation methods (§5.3). For instances where the model failed to improve without external feedback, the LLM judge frequently fails to identify the genuinely corrected response (§5.4). In our naturalistic experiments, for example, the judge correctly preferred the response that solved the problem in only 34% – 54% of the cases. Consequently, the evaluation becomes systematically biased against the feedback signal, penalizing genuine corrections in favor of general stylistic changes (§6.2).

2 Method

We examine whether user feedback provides a useful signal. To simulate a clean scenario where we can easily verify whether the targeted issue was resolved, we couple the naturalistic data experiments with synthetic data experiments.

2.1 Synthetic Data Compilation

To compose our synthetic data, we take tuples of user queries and model responses, and “corrupt” the responses to make them invalid. We then examine whether simulated user feedback is useful for recovering from the corruption.

2.1.1 Response Corruption

We define four corruption types:

- **Causality Inversion:** Swaps the cause and effect in a causal relationship from the reference response, producing a fluent but backward claim. *Example:* “The high inflation rate forced the central bank to raise interest rates” becomes “The central bank raising interest rates forced the high inflation rate.”
- **Crucial Omission:** Deletes a detail that is essential for the response to be correct, safe, or functional, smoothing over the surrounding text

so the gap is not immediately obvious.

Example: “Execute DELETE FROM users WHERE status = ‘inactive’ to clear out the old accounts” becomes “Execute DELETE FROM users to clear out the old accounts.”

- **Entity/Subject Swap:** Replaces a key entity (e.g., a person, location, number, library, or variable name) with a related but factually incorrect one, while keeping the sentence structure and tone unchanged.

Example: “Import the pandas library to manipulate the dataframe” becomes “Import the numpy library to manipulate the dataframe.”

- **Logic Operator Reversal:** Inverts a logical relationship in the response, such as a comparison operator, boolean value, conditional, or chronological order, so the text or code still reads naturally but the underlying logic is flipped.

Example: “Proceed with the installation only if is_admin == True” becomes “Proceed with the installation only if is_admin == False.”

Given the user query q , the original model response r and a corruption type, we prompt a model M_c (henceforth, the *corrupting model*) to corrupt the model response according to the corruption type, making minimal changes. The corrupting model is asked to provide a new corrupted response r_c , a change log c_{log} explaining the corruption, a difficulty score $s_d \in \{LOW, MEDIUM, HIGH\}$ to indicate how hard it is for a generic reader to spot the error, and three levels “user feedback” $\{f_{sol}, f_{prob}, f_{binary}\}$, the first level telling how to fix, the second only pointing out the problem, and the third only saying that the model is wrong (see §B.1 for the full prompts). Formally, we have

$$M_c(q, r, c_type) = (r_c, c_{log}, f_{sol}, f_{prob}, f_{binary})$$

2.2 Naturalistic Data Extraction

To collect real user feedback data, we follow the feedback extraction method of Don-Yehiya et al. (2025b). We prompt *Qwen3-30B-A3B-Instruct-2507* to recognize spans of naturally occurring feedback and classify them into a taxonomy. The taxonomy contains 4 negative categories (*Repeat or Rephrase*, *Make Aware with Correction*, *Make Aware without Correction*, *Ask for Clarification*) and one positive (*Positive Feedback*). We filter out the positive feedback, as in our experiments

we focus on responses improvements (see Appendix B.2).

Manually examining a sample of extracted feedback examples, we find that some instances that have been recognized as feedback, contain feedback that is subjective. Although these feedback instances might be useful for personalization, they should not be used for general alignment training. For example, a user asks for a pasta recipe, and then later provides feedback saying that it should be vegan (*Make Aware with Correction*).

To overcome this, we prompt a model, *Gemini-3-Flash*, to decide whether a given feedback contains actionable insights that should apply to all future users asking the same question or not, see Appendix B.2 for the full prompt.

2.3 Improvement with and without Feedback

Following Liu et al. (2025), we prompt a model M_i , the “improver”, to improve the corrupted response with and without feedback $f \in \{f_{sol}, f_{prob}, f_{binary}\}$, for simplicity we denote it with f . We end up with two improved responses, $M_i(q, r_c, f) = r_{+f}$ and $M_i(q, r_c) = r_{-f}$ accordingly. See §B.3 for the full prompts.

3 Evaluation

3.1 Issue Resolution Evaluation

To examine whether the improved responses r_{+f} and r_{-f} indeed recovered the corruption, we prompt a judge model J_{IR} to assess whether the improved responses addressed f_{sol} , i.e., whether they addressed the simulated feedback that explains how to fix the issue. Thus, we confirm that the improved responses fixed the corruption correctly. The judge is provided with q, r_c, f_{sol} and an improved response (r_{+f} or r_{-f}). The judge is instructed to summarize the feedback intent, compare the changes between the corrupted response and the improved one, check for regression, and finally provide a score (1 is poor 5 is perfect) and a binary decision whether the improved response addressed the feedback or not (see Appendix C). We use this measure also for the naturalistic data. Note that in this case, we do not have the guaranty that addressing the feedback would solve the issue. For example, *Make Aware without Correction* feedback will not provide us with the sufficient information to verify the revision correction. We therefore use this for the naturalistic data as an approximation only.

3.2 Pairwise Evaluation

Given the user query q , we use a judge model J_P to compare two responses directly, response A and response B , to determine which response is the best. For example, we compare the original response and the corrupted one $J_P(q, r, r_c)$, or the two improvements variants $J_P(q, r_{+f}, r_{-f})$. The judge outputs a reasoning explaining its choice, and a verdict $\in \{A, B, Tie\}$. To avoid biases we set the order of the responses randomly. See Appendix C for the full evaluation prompt.

4 Experimental Setup

Data. For the synthetic setting, we use *Arena-Hard-v2.0* (Li et al., 2024). This dataset contains a *hard prompt* category, comprising 500 challenging real-world user queries (open-ended software engineering problems, math questions, logic puzzles, etc.), and a *creative writing* category, comprising 250 creative writing queries sourced from Chatbot Arena (Chiang et al., 2024). We use the model responses of the *o3* model, which is considered to be strong, as we assume that prior to the corruption the response was valid. For naturalistic experiments, we use data from the ShareLM collection (Don-Yehiya et al., 2025c), a collection of open human-model interaction datasets. We filter for English conversations only, for annotations reasons.

Models. For M_c we use *Gemini-3-flash-preview* (Team et al., 2023). For the **improver model** we experiment with two models, differing in size: we use *Gemini-3-flash-preview*, marked as M_i^{large} , and *Qwen3-8B* (Yang et al., 2025), marked as M_i^{small} . For the **judge model** we use a stronger model, *Gemini-3.1-Pro-preview*. As detailed in §H, the multi-stage inference required for corruption generation, improvement, and evaluation incurs substantial computational expense. We therefore restrict our evaluation to this representative set of models. However, we report initial results for *GPT-OSS-20B* in Appendix G, demonstrating consistent trends across architectures.

Corruption Verification. To verify that the corruption process successfully rendered the new responses invalid, we manually evaluated 96 corrupted samples: 64 samples from the hard prompt category and 32 from the creative writing category. See Appendix E for the detailed annotation guidelines and procedures. Out of the hard prompt sam-

Improver	Judge Preference	Causality Inversion	Crucial Omission	Entity Swap	Operator Reversal	Average
Large	w. Feedback	28.5%	28.7%	25.9%	25.8%	27.2%
	w/o. Feedback	36.9%	37.5%	38.7%	37.9%	37.8%
	Tie	34.6%	33.8%	35.4%	36.3%	35.0%
Small	w. Feedback	45.7%	50.0%	52.4%	56.4%	51.2%
	w/o. Feedback	39.2%	37.4%	30.0%	31.2%	34.3%
	Tie	15.1%	12.6%	17.7%	12.5%	14.5%

Table 1: Judge pairwise evaluation of the with-feedback vs. without-Feedback variants. Although the without-feedback variant fix the corruption in a lower rate, we see a consistent preference for the without-feedback variant for the large improver, across the four corruption types.

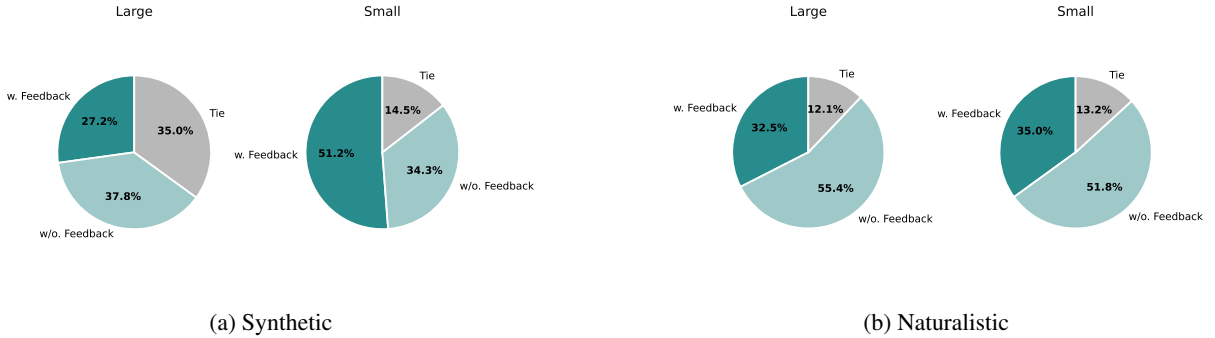


Figure 2: Judge pairwise preference evaluation for with-feedback vs. without-feedback variants. Notably, judges frequently prefer the without-feedback variants despite those variants fixing corruptions at a lower rate.

ples, 92.2% were marked as correct (indicating the corrupted response was successfully invalidated), 3.1% as borderline, 1.6% as incorrect, and 3.1% as unfamiliar (where annotators lacked the domain expertise to evaluate the question). For the creative writing category, 59.4% of the samples were marked as correct, and the rest 40.6% as incorrect. Following these results, to ensure a clean evaluation, we report results exclusively for the hard prompt category in the main text unless otherwise specified. The comprehensive results, including the creative writing category, are provided in Appendix A.

Feedback Relevance Verification. Filtering the naturalistic data to include only feedback cases that should be applied to all future users (see §2.2), the model deemed 47.9% of the naturally occurring feedback cases as relevant. To verify the model’s decisions, we ask a human annotator to review 165 samples of model verdicts and reasoning, see Appendix E for the full prompt. 86.1% of the verdicts were marked as correct (the model was right to tag the feedback as relevant/irrelevant), 7.3% as incorrect, and 6.7% as unfamiliar.

Issue Resolution Verification. To validate our issue resolution evaluation method, we manually annotated 110 samples from the synthetic data (hard prompt category only) and 65 samples from the naturalistic data. For the synthetic data, excluding 16 samples that were marked as unfamiliar, the remaining annotations showed Cohen’s Kappa of 0.81. For the naturalistic data, excluding 2 samples that were marked unfamiliar, we get Cohen’s Kappa of 0.24. See Appendix E for the full guidelines.

5 Results

We generate r_{+f} and r_{-f} for 2000 (500 hard prompt samples $\times$ 4 corruption types) synthetic samples, and 1000 naturalistic data samples (§2.3).

5.1 The Value of External Feedback

To evaluate the overall impact of external feedback, we use the issue resolution evaluation (§3.1) to assess whether the improved responses successfully solved the issues. We compare the with-feedback variant, $J_{FA}(q, r_c, r_{+f}, f)$, against the without-feedback variant, $J_{FA}(q, r_c, r_{-f}, f)$.

Figure 1 presents the issue resolution rates for

both synthetic and naturalistic data. We find that **external feedback consistently helps fix and improve responses, successfully addressing issues in 9%–35% more cases than when feedback is withheld**. Furthermore, while smaller models resolve issues independently at a much lower rate, they benefit far more substantially from the inclusion of feedback than larger models.

In the synthetic setting, the large improver successfully incorporated feedback 95.6% of the time (compared to 86.3% without feedback), while the small improver achieved 77.9% (compared to just 42.7% without). The naturalistic data confirms these trends but, as expected, proves to be a more challenging and noisy environment. Without feedback, the models independently resolved only 70% (large) and 26% (small) of the cases. With feedback, their issue resolution rates jumped to 89% and 53%, respectively. While the absolute issue resolution rates are lower in the naturalistic setting compared to the synthetic one, the performance gain provided by the feedback remains robust.

5.2 The Pairwise Evaluation Discrepancy

Following previous works that sought to quantify the benefit of naturally occurring feedback (Liu et al., 2025; Jin et al., 2025), we compare the feedback-enhanced responses against the without-feedback baseline.

We conduct an LLM-as-a-Judge pairwise comparison between the two improved responses: r_{+f} and r_{-f} (§3.2). Fig. 2 presents the results for both the synthetic and naturalistic data. We find that **although the without-feedback variants resolve the targeted issues at a lower rate, in most settings the pairwise judge consistently prefers them over their with-feedback counterparts**, in line with previous works finding.

For the synthetic data, the judge prefers the without-feedback improvements of the large improver over the with-feedback improvements. This trend is reversed for the small improver, where the with-feedback improvements prove superior. We address this disparity in §6.1. Nevertheless, Table 1 shows that these results are consistent across all four corruption types.

For the naturalistic data, the judge prefers the without-feedback variants generated by both the large and small improvers.

Importantly, this reveals more than a mere lack of improvement due to the feedback. If the improver simply failed to benefit from the feedback,

we would expect a balanced judge to be indifferent between r_{+f} and r_{-f} . Instead, we find that responses with recourse to feedback are judged to be of poorer quality than the baselines ones.

5.3 With-Feedback Should Win

While the previous section showed a discrepant trend between the JLM output and the ground truth, we note that preferring r_{-f} , even where feedback is helpful, is not necessarily erroneous. For example, there are cases where the improver manages to fix the corruption by itself, without feedback (§5.1). For such cases, there is no reason to prefer the with-feedback variant. Moreover, it is possible that these without-feedback variants that solved the issue also improved other aspects of the response and should rightfully be preferred.

We therefore recompute the evaluation only on the subset of user queries where r_{+f} successfully addressed the issue while r_{-f} failed. Formally, this is defined as

$$\begin{aligned} \text{WFShouldWin}(M_i) = & \{q \mid J_{FA}(q, r_c, r_{+f}, f) = \text{True}\} \\ & \cap \{q \mid J_{FA}(q, r_c, r_{-f}, f) = \text{False}\} \end{aligned}$$

On this subset, a judge is correct iff it prefers r_{+f} , as it is the only one that resolves the corruption. Filtering for $q \in \text{WFShouldWin}$ for the various settings, we end up with 205 and 574 synthetic examples, and 215 and 274 naturalistic examples for M_i^{large} and M_i^{small} respectively.

The pairwise results highlights a frequent limitation in the automated evaluation of feedback signals: **even when filtering for examples where r_{+f} should win, the judge often fails in doing so**.

For the synthetic data, we find that the judge correctly prefers the with-feedback variant of the small improver in 80.9% of the cases. For the large improver however, the judge correctly prefers the with-feedback variant in only 54.6% of the cases, i.e there are 45.4% cases where the judge makes an error and fails to prefer a valid response over an invalid one. Similarly, in the natural data the judge correctly prefers the with-feedback variants in only 34% of the cases for the large improver, and 54% of the cases for the small improver.

5.4 Models Struggle to Evaluate What They Cannot Improve

Following the previous judge failure, we hypothesize that a model’s limitations as an improver correlate with its limitations as an evaluator. Specifically,

we suspect that a model will tend to fail as a judge on queries where it failed to self-improve.

To test this, we compare the judge’s accuracy on two sets of queries: those M_i^{large} can self-improve, and those it cannot. The methodological challenge here is establishing a clear ground truth for the judgment task. If we were to use M_i^{large} ’s own outputs for this pairwise comparison, the queries where it successfully resolved the corruption both with and without feedback would yield valid responses. Therefore, it would not be possible to determine on what cases the judge was correct. Manually annotated ground truth is also infeasible (see §8).

To resolve this, we decouple the query split from the evaluated responses by leveraging M_i^{small} . We restrict our evaluation entirely to the $WFShouldWin(M_i^{small})$ subset of the synthetic data. For these queries, we have a clear ground truth: the small improver’s r_{+f} successfully resolves the corruption, while its r_{-f} fails. Therefore, in a pairwise comparison of these examples, r_{+f} should always win.

We partition these ground-truth queries based on how the large improver performed on them, creating two subsets:

- **The self-improvable subset:** Queries where M_i^{large} successfully resolved the corruption with or without feedback.

$$\text{Both}(M_i^{large}) = \{q \mid J_{FA}(q, r_c, r_{+f}, f) = \text{True}\} \\ \cap \{q \mid J_{FA}(q, r_c, r_{-f}, f) = \text{True}\}$$

- **The feedback-improvable subset:** Queries where M_i^{large} could *only* resolve the corruption when provided with feedback, namely $WFShouldWin(M_i^{large})$.

We then task M_i^{large} to act as a judge, running a pairwise comparison between r_{+f} and r_{-f} generated by M_i^{small} for both subsets. The M_i^{large} judge correctly prefers r_{+f} at high rates for both subsets: 87.7% on the self-improvable subset, and 72.2% on the feedback-improvable subset. However, its accuracy is 15.5 percentage points lower on the feedback-dependent subset. This supports our hypothesis: **models exhibit correlated failure modes: an inability to independently improve a corrupted response corresponds to a decreased accuracy in evaluating it.**

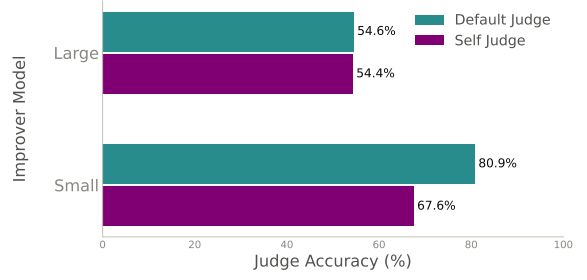


Figure 3: Judge accuracy for the large and small improvers, with both default and self judges. While for the large improver there is no significant gap between the default and self judge, for the small improver with self judge we see for the first time that the judge struggle to prefer the correct response (the with-feedback variant).

6 Further Analysis

6.1 Reproducing Correlated Failures in the Small Improver

To further investigate the relationship between a model’s capabilities as an improver and as an evaluator, we assess the models in a self-judging setup where the improver also acts as the pairwise judge.

Figure 3 compares the evaluation accuracy of the default judge (*Gemini-3.1-Pro*) against the respective self-judges for the $WFShouldWin(M_i^{large})$ and $WFShouldWin(M_i^{small})$ subsets (§5.3) of the synthetic data.

This self-judging setup explains the different trends we saw for the large and small improvers. When using the default judge to evaluate the small model’s subset, the evaluation appeared highly accurate (§5.3). This occurred because of the substantial capability gap between the two models: issues that were hard for *Qwen3-8B* were still relatively easy for *Gemini-3.1-Pro*. Conversely, the capability gap between the default judge and the large improver is significantly narrower. Thus, the issues that *Gemini-3-Flash* failed to resolve were also difficult for *Gemini-3.1-Pro* to judge, making the shared limitations immediately visible during that comparison. Without the advantage of a more capable judge, misjudgment rates spike significantly, and we finally observe the same correlated failure trends previously seen with the large model.

The data confirms this dynamic. For the large improver, the default judge and the self-judge correctly prefer r_{+f} at practically the same rate: 54.6% and 54.4% respectively. However, when we use *Qwen3-8B* as a self-judge on its respective subset, a substantial performance gap emerges. While

the default judge prefers the with-feedback variant in 80.8% of the cases, the self-judge accuracy is only 67.6% correct, 13 points lower.

6.2 Improvement Types

How do improvements with and without feedback differ? This comparison may explain why the judge prefers still-corrupted responses.

We prompt a model to classify the $WFShouldWin(M_i^{large})$ attempted improvements of the full synthetic dataset (hard prompt + creative writing). The model categorizes each response into a *No Improvement* option, or into one of six categories across two dimensions: *Content* (Factuality, Completeness, Logic) and *Style* (Tone, Conciseness, Formatting and Structure). See Appendix D for the full prompt.

Figure 4 demonstrates that providing feedback increases the proportion of content improvements relative to style improvements. Additionally, the number of “no improvement” cases decreases when feedback is provided.

We conclude that **feedback yields higher-quality improvements focused on substantive content changes rather than stylistic edits. The pairwise judge, however, fails to recognize this and is biased towards stylistic changes.**

See Appendix F for further evidence of judge bias towards stylistic edits.

Improver	w/o. Feedback	Full Feedback	What Problem	Problem Exists
Large	86.3%	95.6%	92.8%	89.7%
	–	$\Delta+9.3\%$	$\Delta+6.3\%$	$\Delta+3.4\%$
Small	49.2%	80.8%	65.85%	59.5%
	–	$\Delta+31.6\%$	$\Delta+16.7\%$	$\Delta+10.3\%$

Table 2: Percentage of improved responses successfully addressing feedback, with different levels of feedback signal. Although the “Full Feedback” signal yields the largest delta (9% – 32%), the weaker signals are still beneficial for the improvers.

6.3 Weaker Feedback Signal

Our synthetic data experiments used feedback that tells the model how to fix the corruption (f_{sol}). Here we examine weaker feedback signals, that only point out the problem (f_{prob}), or just state that a problem exists (f_{binary}).

We generate $M_i(q, r_c, f_{prob}) = r_{+f_{prob}}$ and $M_i(q, r_c, f_{binary}) = r_{+f_{binary}}$ for 100 queries for each corruption type (400 for each, 800 total), and

examine whether they better address the feedback over the without-feedback variant r_{-f} . Note that we provide J_{IR} with f_{sol} (and not f_{prob} / f_{binary} accordingly) as we want to provide it with sufficient information to verify whether the corruption was solved, thus we have $J_{FA}(q, r_c, f_{sol}, r_{+f_{prob}})$ and $J_{FA}(q, r_c, f_{sol}, r_{+f_{binary}})$.

Table 2 demonstrates that both the feedback that points to the problem (“What Problem”), and the feedback that only states that there is a problem (“Problem exists”) solve the corruption in a higher rate than the no-feedback variant. However, as expected, their margin is smaller. While for the full feedback (§5.1) the improvers resolved about 9% – 32% more cases than without any feedback, “What Problem” feedback improves about 6% – 17% more cases, and “Problem exists” improves only about 3% – 10% more cases. Also, similar to the full feedback, the small improver benefits more from the feedback.

We conclude that pointing to a problem, or just indicating that a problem exists are valuable, even if less than the full feedback.

7 Related Work

A related line of work emphasized the utility of *natural language feedback* (Hancock et al., 2019; Yan et al., 2023; Jayalath and Ramaswamy, 2022; Sreedhar et al., 2020). In such works, the user is asked to provide free-text feedback (unlike the more standard binary feedback or other close forms). Later works (Don-Yehiya et al., 2025b; Lin et al., 2024; Shi et al., 2026) take a step further and look for feedback that spontaneously occurs in the conversations, without explicitly asking the user to provide it, and use it for model training (Buening et al., 2026; Jin et al., 2025).

Liu et al. (2025) studied the question of whether naturally occurring user feedback can indeed improve model performance. They used a large model to create “regeneration with semantics” responses that used human feedback, and “regeneration from scratch” responses as a baseline, similar to our with-feedback vs. without-feedback responses. Using a reward model to compare them against each other, they found that the regenerations from scratch outperformed the semantics ones. This is in line with our results, that for the large improver the without-feedback responses outperformed the with-feedback responses (§5.2). However, our analysis revealed that this is due to a problem in the evalua-

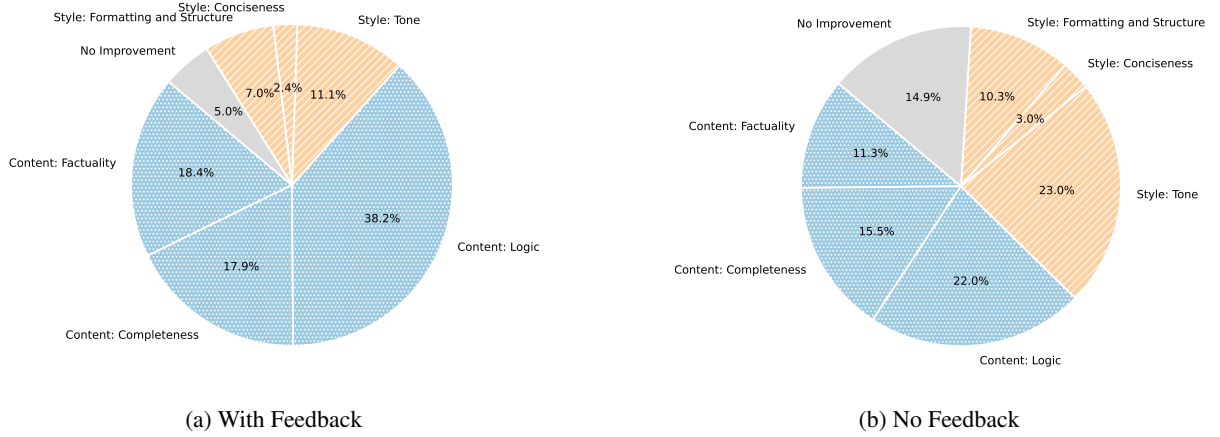


Figure 4: Distribution of improvement types applied to corrupted model responses. Content-related improvements (factuality, completeness, and logic) are represented in shades of blue with a dotted texture. Style-related improvements (tone, conciseness, formatting) are represented in shades of orange with a diagonal line texture. Providing feedback significantly increases the proportion of substantive content improvements compared to stylistic edits.

tion, not the signal’s usefulness (§5.3).

In our work we demonstrated how the well used "LLM as a Judge" evaluation (Chiang and Lee, 2023; Liu et al., 2023; Zhu et al., 2023) fails to prefer the valid response over the corrupted one. Previous works already show that this method suffers from biases, such as position, verbosity and self-preference (Wang et al., 2024; Saito et al., 2023; Koo et al., 2024; Don-Yehiya et al., 2026).

In the scope of this work we improve the model in test-time (Madaan et al., 2023), without further training. Another line of work investigates self-improve the model via training (Zweiger et al., 2025; Huang et al., 2023; Yuan et al., 2024).

8 Discussion

Through validation on both synthetic and naturalistic data, we established that while LLMs can resolve numerous issues autonomously, incorporating user feedback enables them to resolve 9% – 32% more cases compared to baselines without feedback. However, this improvement is not captured during evaluation: despite a higher resolution rate when feedback is applied, pairwise evaluations frequently favor the without-feedback variants. This explains the prevailing misconception that feedback does not provide a useful signal.

Preference for the without-feedback variant may be warranted in instances where both variants successfully resolve the error, as the without-feedback variant may introduce other general improvements (§6.2). However, in cases where only r_{+f} variant resolves the corruption, a preference for r_{-f} or a

tie, is a clear error on the LLM judge’s behalf.

Further analysis reveals a circular pattern: the judge frequently fails to correctly judge cases that the LLM improver failed to improve on its own. This yields a bias against the feedback signal.

We speculate that this systematic bias may be, oddly enough, evidence for the utility of naturally occurring feedback. The fact that LLM judges fail to recognize feedback-informed improvements might suggest that the feedback provides a unique signal that is not encoded in the model’s weights. Thus, it is reasonable to assume that it can not be acquired through standard techniques such as model distillation or iterative self-improvement.

Although demonstrated in an inference-only setting, we expect our findings to persist in training as well. Consequently, future work should prioritize the development of more robust evaluation frameworks that accurately reflect the utility of feedback. We urge the community to look beyond standard LLM judgment paradigms and explore specialized reward models, interactive evaluation metrics, or targeted alignment strategies designed specifically to break this circular bias. Addressing this fundamental evaluation gap is critical; without reliable mechanisms to appropriately credit feedback-driven resolutions, we risk artificially impeding the development of open models, developed with open user data.

Limitations

Our manual annotation revealed that the corruption process is less effective for the creative writing

category of our synthetic data.

Therefore, to ensure reliability, the main text focuses exclusively on the math and code domains. However, results for the full synthetic dataset, including creative writing, are available in the appendix (§A). The overall trends persist, which, along with our multi-domain naturalistic data, indicate that our findings generalize beyond math and code domains.

To ensure reliability, we manually annotated representative samples from all automated stages of our pipeline: the corruption process, feedback relevancy filtering, and feedback addressing evaluation. However, we did not manually annotate the pairwise evaluation setting, as that proved to be prohibitively difficult for humans. We did, however, conduct a small-scale human annotation task which confirmed that manual pairwise evaluation is highly challenging. The sheer length of the responses makes it difficult to track minute details, and the diverse topics necessitate broad domain expertise.

Acknowledgments

This research was partly supported by a grant from the Israel Science Foundation (grant no. 2912/25). Special thanks to our annotators, Nicole Gruber and Yoav Schmidt.

References

- Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K Arora, Yu Bai, Bowen Baker, Haiming Bao, and 1 others. 2025. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*.
- Mohammad Amin Bassiri. 2011. Interactional feedback and the impact of attitude and motivation on noticing 12 form. *English Language and Literature Studies*, 1(2):61.
- Janet Beavin Bavelas and Jennifer Gerwing. 2011. [The listener as addressee in face-to-face dialogue](#). *International Journal of Listening*, 25(3):178–198.
- Thomas Kleine Buening, Jonas Hübotter, Barna Pásztor, Idan Shenfeld, Giorgia Ramponi, and Andreas Krause. 2026. Aligning language models from user interactions. *arXiv preprint arXiv:2603.12273*.
- Cheng-Han Chiang and Hung-yi Lee. 2023. [Can large language models be an alternative to human evaluations?](#) In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15607–15631, Toronto, Canada. Association for Computational Linguistics.
- Wei-Lin Chiang, Lianmin Zheng, Ying Sheng, Anastasios Nikolas Angelopoulos, Tianle Li, Dacheng Li, Hao Zhang, Banghua Zhu, Michael Jordan, Joseph E Gonzalez, and 1 others. 2024. Chatbot arena: An open platform for evaluating llms by human preference. *arXiv preprint arXiv:2403.04132*.
- Shachar Don-Yehiya, Ben Burtenshaw, Ramon Fernandez Astudillo, Cailean Osborne, Mimansa Jaiswal, Tzu-Sheng Kuo, Wenting Zhao, Idan Shenfeld, Andi Peng, Mikhail Yurochkin, and et al. 2025a. [The future of open human feedback](#). *Nature Machine Intelligence*, 7(6):825–835.
- Shachar Don-Yehiya, Leshem Choshen, and Omri Abend. 2025b. [Naturally occurring feedback is common, extractable and useful](#). *Preprint*, arXiv:2407.10944.
- Shachar Don-Yehiya, Leshem Choshen, and Omri Abend. 2025c. [The ShareLM collection and plugin: Contributing human-model chats for the benefit of the community](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, pages 167–177, Vienna, Austria. Association for Computational Linguistics.
- Shachar Don-Yehiya, Asaf Yehudai, Leshem Choshen, and Omri Abend. 2026. [Mediocrity is the key for LLM as a judge anchor selection](#). In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15491–15513, San Diego, California, United States. Association for Computational Linguistics.
- Braden Hancock, Antoine Bordes, Pierre-Emmanuel Mazare, and Jason Weston. 2019. [Learning from dialogue after deployment: Feed yourself, chatbot!](#) In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 3667–3684, Florence, Italy. Association for Computational Linguistics.
- Jiaxin Huang, Shixiang Gu, Le Hou, Yuexin Wu, Xuezhi Wang, Hongkun Yu, and Jiawei Han. 2023. [Large language models can self-improve](#). In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 1051–1068, Singapore. Association for Computational Linguistics.
- Hemadri Jayalath and Lakshmish Ramaswamy. 2022. [Enhancing performance of operationalized machine learning models by analyzing user feedback](#). In *Proceedings of the 2022 4th International Conference on Image, Video and Signal Processing, IVSP ’22*, page 197–203, New York, NY, USA. Association for Computing Machinery.
- Chuanyang Jin, Jing Xu, Bo Liu, Leitian Tao, Olga Golovneva, Tianmin Shu, Wenting Zhao, Xian Li, and Jason Weston. 2025. The era of real-world human interaction: RL from user conversations. *arXiv preprint arXiv:2509.25137*.

- Ryan Koo, Minhwa Lee, Vipul Raheja, Jong Inn Park, Zae Myung Kim, and Dongyeop Kang. 2024. [Benchmarking cognitive biases in large language models as evaluators](#). In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 517–545, Bangkok, Thailand. Association for Computational Linguistics.
- Tianle Li, Wei-Lin Chiang, Evan Frick, Lisa Dunlap, Tianhao Wu, Banghua Zhu, Joseph E Gonzalez, and Ion Stoica. 2024. From crowdsourced data to high-quality benchmarks: Arena-hard and benchbuilder pipeline. *arXiv preprint arXiv:2406.11939*.
- Ying-Chun Lin, Jennifer Neville, Jack Stokes, Longqi Yang, Tara Safavi, Mengting Wan, Scott Counts, Siddharth Suri, Reid Andersen, Xiaofeng Xu, Deepak Gupta, Sujay Kumar Jauhar, Xia Song, Georg Buscher, Saurabh Tiwary, Brent Hecht, and Jaime Teevan. 2024. [Interpretable user satisfaction estimation for conversational systems with large language models](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11100–11115, Bangkok, Thailand. Association for Computational Linguistics.
- Yang Liu, Dan Iter, Yichong Xu, Shuhang Wang, Ruochen Xu, and Chenguang Zhu. 2023. [G-eval: NLG evaluation using gpt-4 with better human alignment](#). In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 2511–2522, Singapore. Association for Computational Linguistics.
- Yuhan Liu, Michael JQ Zhang, and Eunsol Choi. 2025. [User feedback in human-LLM dialogues: A lens to understand users but noisy as a learning signal](#). In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 2666–2681, Suzhou, China. Association for Computational Linguistics.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, and 1 others. 2023. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems*, 36:46534–46594.
- Martin J. Pickering and Simon Garrod. 2021. *Understanding Dialogue: Language Use and Social Interaction*. Cambridge University Press.
- Keita Saito, Akifumi Wachi, Koki Wataoka, and Youhei Akimoto. 2023. Verbosity bias in preference labeling by large language models. *arXiv preprint arXiv:2310.10076*.
- Taiwei Shi, Zhuoer Wang, Longqi Yang, Ying-Chun Lin, Zexue He, Mengting Wan, Pei Zhou, Sujay Kumar Jauhar, Sihao Chen, Shan Xia, Hongfei Zhang, Jieyu Zhao, Xiaofeng Xu, Xia Song, and Jennifer Neville. 2026. [WildFeedback: Aligning LLMs with in-situ user interactions and feedback](#). In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 36701–36725, San Diego, California, United States. Association for Computational Linguistics.
- Makesh Narsimhan Sreedhar, Kun Ni, and Siva Reddy. 2020. [Learning improvised chatbots from adversarial modifications of natural language feedback](#). In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 2445–2453, Online. Association for Computational Linguistics.
- Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, and 1 others. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*.
- Jelena Vranjes, Geert Brône, and Kurt Feyaerts. 2018. [Dual feedback in interpreter-mediated interactions: On the role of gaze in the production of listener responses](#). *Journal of Pragmatics*, 134:15–30.
- Peiyi Wang, Lei Li, Liang Chen, Zefan Cai, Dawei Zhu, Binghuai Lin, Yunbo Cao, Lingpeng Kong, Qi Liu, Tianyu Liu, and Zhifang Sui. 2024. [Large language models are not fair evaluators](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9440–9450, Bangkok, Thailand. Association for Computational Linguistics.
- Margaret G. Werts, Mark Wolery, Ariane Holcombe, and David L. Gast. 1995. [Instructive feedback: Review of parameters and effects](#). *Journal of Behavioral Education*, 5(1):55–75.
- Hao Yan, Saurabh Srivastava, Yintao Tai, Sida I. Wang, Wen-tau Yih, and Ziyu Yao. 2023. [Learning to simulate natural language feedback for interactive semantic parsing](#). In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3149–3170, Toronto, Canada. Association for Computational Linguistics.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, and 1 others. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.
- Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Xian Li, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. 2024. Self-rewarding language models. In *Proceedings of the 41st International Conference on Machine Learning, ICML’24*. JMLR.org.
- Lianghui Zhu, Xinggang Wang, and Xinlong Wang. 2023. Judgelm: Fine-tuned large language models are scalable judges. *arXiv preprint arXiv:2310.17631*.
- Adam Zweiger, Jyo Pari, Han Guo, Yoon Kim, and Pulkit Agrawal. 2025. [Self-adapting language models](#). In *Advances in Neural Information Processing Systems*, volume 38, pages 74084–74115. Curran Associates, Inc.

	Causality Inversion	Crucial Omission	Entity Swap	Operator Reversal
Original	96.6%	98.7%	99.4%	99.2%
Corrupted	0.9%	1.1%	0.4%	0.8%
Tie	2.6%	0.2%	0.2%	0.0%

Table 3: Judge Comparison: Original vs. Corrupted Responses

A Full Results

We provide here the full results, including the creative writing category.

We use the judge model J_{IR} (§3.2) to compare the original response r and the corrupted one r_c . Table 3 confirms that in this clean setup, with minimal changes introduced by the corruption model, the judge correctly prefers the original response.

Table 4 presents the issue resolution rates for the hard prompt category (synthetic data). Table 5 present the issue resolution rates for the full synthetic data, i.e., hard prompt + creative writing. We see that the trends remain the same, with a consistent positive delta between the without-feedback and with-feedback variants.

Table 3 presents the judge pairwise preferences results for the hard prompt category of the synthetic data. Table 6 presents the pairwise results for the full synthetic data. Also here, the trends remain the same, with even larger effect (the preference of the without-feedback variant is stronger).

Table 7 presents the judge pairwise preferences results for the real user data. Table 8 presents the results for the *WFSouldWin* subset of the user data.

Fig. 19 presents the improvement type results for the hard prompt category.

B Data Extraction

B.1 Corruption Prompts

We prompt Gemini-3-flash-preview with the corruption prompts detailed in Figures 5, 6, 7, and 8.

B.2 Naturally Occurring Feedback Extraction

To identify and categorize user feedback within the conversations, we process full conversations using the extraction prompt detailed in Figure 9. We then filter out all UR5 feedback instances.

To evaluate whether the simulated user feedback contains universally actionable insights or is merely

subjective, we prompt Gemini-3-flash-preview using the template detailed in Figure 14.

B.3 Response Improvement Prompts

To revise and improve the model’s responses, we use two variations of an improvement prompt. Figure 10 is used when specific human feedback is available, while Figure 11 relies solely on the conversation context.

C Evaluation Prompts

C.1 Issue Resolution Prompt

To automatically evaluate whether the revised model responses successfully incorporated the user’s feedback, we use the resolution addressing prompt detailed in Figure 13.

C.2 Pairwise Evaluation Prompt

To evaluate the quality of the revised responses compared to the originals, we use an LLM-as-a-judge approach with the pairwise evaluation prompt detailed in Figure 12.

D Improvement Type Prompt

To categorize the differences between the with-feedback vs. without-feedback improvements, we use the prompt detailed in Figure 15.

E Human Annotation

To confirm the reliability of our automated pipelines, we conducted a human evaluation of the data extraction and evaluation processes.

The tasks were performed by two in-house annotators: an experienced annotator and a specialist with expertise in code and mathematics. Due to the highly diverse nature of the data, achieving comprehensive coverage was unfeasible, and certain examples were consequently labeled as ‘unfamiliar’. Furthermore, owing to the broad range of domains and the granular level of detail, the annotators reported they could not conclusively guarantee that no regressions were introduced.

E.1 Corruption Verification Guidelines

To ensure consistent quality in the human evaluation of our corrupted dataset, annotators were provided with the instructions detailed in Figure 16.

Improver	Variant	Causality Inversion	Crucial Omission	Entity Swap	Operator Reversal	Average
Large	w/o feedback	86.6%	85.0%	85.1%	88.5%	86.3%
	w/ feedback	97.2%	94.9%	94.8%	95.5%	95.6%
		$\Delta+10.6\%$	$\Delta+9.9\%$	$\Delta+9.7\%$	$\Delta+7.0\%$	$\Delta+9.3\%$
Small	w/o feedback	52.6%	47.5%	47.4%	49.4%	49.2%
	w/ feedback	78.9%	79.1%	83.1%	82.1%	80.8%
		$\Delta+26.3\%$	$\Delta+31.6\%$	$\Delta+35.7\%$	$\Delta+32.7\%$	$\Delta+31.6\%$

Table 4: Percentage of improved responses successfully addressing issues for the hard prompt category of the synthetic data. We observe a consistent positive delta between the without-feedback and with-feedback variants across the four corruption types.

Improver	Variant	Causality Inversion	Crucial Omission	Entity Swap	Operator Reversal	Average
Large	w/o feedback	77.4%	80.2%	72.0%	78.9%	77.1%
	w/ feedback	98.0%	96.5%	96.0%	97.0%	96.9%
		$\Delta+20.6\%$	$\Delta+16.3\%$	$\Delta+24.0\%$	$\Delta+18.1\%$	$\Delta+19.8\%$
Small	w/o feedback	43.1%	40.8%	38.7%	39.1%	40.4%
	w/ feedback	72.7%	76.7%	82.4%	75.1%	76.7%
		$\Delta+29.6\%$	$\Delta+35.9\%$	$\Delta+43.7\%$	$\Delta+36.0\%$	$\Delta+36.3\%$

Table 5: Percentage of improved responses successfully addressing issues. We observe a consistent positive delta between the without-feedback and with-feedback variants across the four corruption types.

Improver	Judge Preference	Causality Inversion	Crucial Omission	Entity Swap	Operator Reversal	Average
Large	w. Feedback	29.5%	29.6%	26.1%	26.5%	27.9%
	w/o. Feedback	41.2%	41.7%	45.0%	43.4%	42.8%
	Tie	29.3%	28.7%	29.0%	30.1%	29.3%
Small	w. Feedback	49.1%	53.8%	56.8%	56.3%	54.0%
	w/o. Feedback	38.4%	36.5%	28.0%	32.6%	33.9%
	Tie	12.6%	9.8%	15.2%	11.1%	12.2%

Table 6: Judge pairwise evaluation of the with-feedback vs. without-feedback variants on the full synthetic data (hard prompt + creative writing). Although the without-feedback variant fixes the corruption at a lower rate, we see a consistent preference for the without-feedback variant for the large improver. The hard prompt only results are in Fig. 1

Improver	Judge Preference		
	w. Feedback	w/o. Feedback	Tie
Large	32.5%	55.4%	12.1%
Small	35.0%	51.8%	13.2%

Improver	<i>N</i>	Judge Preference		
		w. Feedback	w/o. Feedback	Tie
Large	215	34.0%	62.8%	3.3%
Small	274	54.0%	40.5%	5.5%

Table 7: Judge pairwise evaluation of the with-feedback vs. without-feedback variants, for the real-user data. We see similar trends to those of the synthetic data: although the without-feedback variants fix the corruption in a lower rate, we see a consistent preference for the without-feedback variants, here not only for the large improver but also for the small improver.

Table 8: Judge preference for the *WFShouldWin* subset (where No-Feedback failed to address feedback but With-Feedback succeeded), on real user data.

E.2 Relevance Verification Guidelines

To evaluate the performance of our evaluator in distinguishing universal from subjective feedback, human annotators were provided with the instructions detailed in Figure 17.

Corruption Prompt 1: Causality Inversion

System Role: You are an expert data evaluator and adversarial example generator.

Objective: Take the provided [Question] and [Reference Response], generate a [Corrupted Response] using a Causality Inversion, and then generate simulated User Feedback correcting the error.

Rules:

1. Identify a cause-and-effect relationship described in the response.
2. Swap the cause and the effect, making the claim backward. Ensure the new sentence flows naturally. Do not add disclaimers.
3. Generate three levels of simulated user feedback. The feedback must read naturally as a human chatting with an AI. Keep the feedback short, casual, and slightly lazy, mimicking how a real user would quickly type on a phone or keyboard. Do NOT mention the "corruption process" or the "prompt".

Output Format:

Corrupted Response: [The fully generated text]

Edit Log: [Explain exactly the cause and effect that were swapped]

Detection Difficulty: [Rate as LOW, MEDIUM, or HIGH based strictly on how hard it is for a general reader to spot the error, and provide a 1-sentence justification.]

User Feedback (Level 1 - How to fix): [A short, lazy reply (1-2 sentences max) pointing out the reversed causality and giving the right one (e.g., "Other way around, X causes Y.")]

User Feedback (Level 2 - What is the problem): [A brief reply pointing out it's backward without explicitly fixing it (e.g., "I think you mixed up cause and effect.", "That's backward.")]

User Feedback (Level 3 - Just 'you are wrong'): [A very short, blunt reply simply stating it is wrong (e.g., "Makes no sense.", "Wrong.", "Nope.")]

Figure 5: Prompt for generating causality inversion corruptions.

E.3 Feedback Addressing Guidelines

To evaluate whether the revised model responses successfully incorporated the user feedback, human annotators were provided with the instructions detailed in Figure 18.

F Re-improvements

Another direction we explore is iterative improvement: applying an initial improvement step without feedback, followed by a second step that incorporates feedback. This sequential approach has the potential to combine the best of both worlds, merging the targeted corrections of the with-feedback variant with the broader stylistic enhancements of the without-feedback variant (see §6.2).

We run this analysis for the large improver only. We isolate the subset of queries where the initial with-feedback response (r_{+f}) successfully addressed the feedback, but the no-feedback response

(r_{-f}) did not ($WFShouldWin(M_i^{large})$). Because r_{-f} remains corrupted, we know it requires further refinement. Furthermore, since r_{+f} successfully resolved the issue for these specific queries, there is strong reason to believe that a secondary improvement step utilizing feedback will be effective.

We take r_{-f} and apply a second improvement step—this time using feedback—yielding $M_i(q, r_{-f}, f)$. We first evaluate this new output against the single-step r_{-f} . To ensure a fair comparison regarding the computational resources used, we also introduce a secondary baseline where r_{-f} is improved a second time *without* feedback, denoted as $M_i(q, r_{-f})$.

Table 13 shows that the judge prefers the two-stage with-feedback response ($M_i(q, r_{-f}, f)$) over the single-stage baseline (r_{-f}). However, when comparing the two two-stage methods against each other, the judge's preference reverses, favoring

Corruption Prompt 2: Crucial Omission

System Role: You are an expert data evaluator and adversarial example generator.

Objective: Take the provided [Question] and [Reference Response], generate a [Corrupted Response] using a Crucial Omission, and then generate simulated User Feedback correcting the error.

Rules:

1. Identify a critical detail that is absolutely necessary for the response to be correct, safe, or functional.
2. Delete this detail entirely. Smooth over the surrounding text or code so the omission is not immediately obvious. Do not add disclaimers.
3. Generate three levels of simulated user feedback. The feedback must read naturally as a human chatting with an AI. Keep the feedback short, casual, and slightly lazy, mimicking how a real user would quickly type on a phone or keyboard. Do NOT mention the "corruption process" or the "prompt".

Output Format:

Corrupted Response: [The fully generated text]

Edit Log: [Explain exactly what crucial detail was removed]

Detection Difficulty: [Rate as LOW, MEDIUM, or HIGH based strictly on how hard it is for a general reader to spot the error, and provide a 1-sentence justification.]

User Feedback (Level 1 - How to fix): [A short, lazy reply (1-2 sentences max) noting what's missing and saying what to add (e.g., "You forgot to import json.", "Add the baking powder.")]

User Feedback (Level 2 - What is the problem): [A brief reply stating that it's incomplete without providing the missing info (e.g., "Seems like a step is missing.", "This won't compile, you forgot something.")]

User Feedback (Level 3 - Just 'you are wrong'): [A very short, blunt reply simply stating it is wrong (e.g., "Incomplete.", "Broken.", "Doesn't work.")]

Figure 6: Prompt for generating crucial omission corruptions.

the response improved twice *without* feedback ($M_i(q, r_{-f})$). Even when examining only the subset of cases ($n = 123$) where the two-stage with-feedback approach successfully addressed the feedback but the two-stage without-feedback approach failed to do so, the judge correctly prefers the with-feedback variant in only 34.2% of the cases.

It seems that the judge is heavily biased toward the stylistic polish of the without-feedback variant, often prioritizing general fluency and formatting over strict adherence to the provided feedback.

G Initial Results with Another Improver

We present initial results with *GPT-OSS-20B* (Agarwal et al., 2025) on about 400 synthetic data samples, see tables 14 and 15.

H Computational Budget

We spent about 2,000\$ on the Gemini API (generating corruptions, improvements, relevance evaluation, and judgments).

To run Qwen3-8B we used a 45G GPU (a40 or the like). Generating improvements for 100 samples took about 2.5 hours, so the total generations for the main experiments took about 75 hours. Generating judgements for the self-judge experiments took another 35 hours.

To generate the initial results for GPT-OSS-20B (\$G) we used a h200, for a total of 10 hours.

Corruption Prompt 3: Entity Swap

System Role: You are an expert data evaluator and adversarial example generator.

Objective: Take the provided [Question] and [Reference Response], generate a [Corrupted Response] using an Entity Swap, and then generate simulated User Feedback correcting the error.

Rules:

1. Identify a key entity in the reference response (e.g., a historical figure, a location, a specific number, a coding library, or a variable name).
2. Swap it with a related, plausible, but factually incorrect entity.
3. Keep the rest of the sentence structure and tone completely identical. Do not add disclaimers.
4. Generate three levels of simulated user feedback. The feedback must read naturally as a human chatting with an AI. Keep the feedback short, casual, and slightly lazy, mimicking how a real user would quickly type on a phone or keyboard. Do NOT mention the "corruption process" or the "prompt".

Output Format:

Corrupted Response: [The fully generated text]

Edit Log: [Explain exactly what was swapped]

Detection Difficulty: [Rate as LOW, MEDIUM, or HIGH based strictly on how hard it is for a general reader to spot the error, and provide a 1-sentence justification.]

User Feedback (Level 1 - How to fix): [A short, lazy reply (1-2 sentences max) pointing out the error and providing the fix casually (e.g., "Wait, X is actually Y.")]

User Feedback (Level 2 - What is the problem): [A brief reply pointing out what is wrong without giving the right answer (e.g., "Are you sure about X?", "I think the location is wrong.")]

User Feedback (Level 3 - Just 'you are wrong'): [A very short, blunt reply simply stating it is wrong (e.g., "Wrong.", "Nope.", "That's incorrect.")]

Figure 7: Prompt for generating entity swap corruptions.

Improver	Variant	Causality Inversion	Crucial Omission	Entity Swap	Operator Reversal	Average
Gemini-3-Flash	With-Feedback	91.1%	94.6%	84.4%	93.5%	90.9%
	No-Feedback	84.8%	84.0%	75.0%	83.3%	81.7%
		$\Delta+6.3\%$	$\Delta+10.6\%$	$\Delta+9.4\%$	$\Delta+10.2\%$	$\Delta+9.2\%$

Table 9: Percentage of Responses Successfully Addressing Feedback (Level 2)

Improver	Variant	Causality Inversion	Crucial Omission	Entity Swap	Operator Reversal	Average
Gemini-3-Flash	With-Feedback	79.6%	90.2%	80.6%	89.4%	84.9%
	No-Feedback	84.8%	84.0%	75.0%	83.3%	81.7%
		$\Delta-5.2\%$	$\Delta+6.2\%$	$\Delta+5.6\%$	$\Delta+6.1\%$	$\Delta+3.2\%$

Table 10: Percentage of Responses Successfully Addressing Feedback (Level 3)

Corruption Prompt 4: Logic Reversal

System Role: You are an expert data evaluator and adversarial example generator.

Objective: Take the provided [Question] and [Reference Response], generate a [Corrupted Response] using a Logic Reversal, and then generate simulated User Feedback correcting the error.

Rules:

1. Identify a logical relationship in the reference response (e.g., a mathematical operator, a boolean state, a chronological sequence, or a conditional statement).
2. Invert or reverse this specific logic.
3. Ensure the text or code still reads naturally and compiles/parses conceptually. Do not add disclaimers.
4. Generate three levels of simulated user feedback. The feedback must read naturally as a human chatting with an AI. Keep the feedback short, casual, and slightly lazy, mimicking how a real user would quickly type on a phone or keyboard. Do NOT mention the "corruption process" or the "prompt".

Output Format:

Corrupted Response: [The fully generated text]

Edit Log: [Explain exactly what logic was reversed]

Detection Difficulty: [Rate as LOW, MEDIUM, or HIGH based strictly on how hard it is for a general reader to spot the error, and provide a 1-sentence justification.]

User Feedback (Level 1 - How to fix): [A short, lazy reply (1-2 sentences max) pointing out the logical error and providing the fix (e.g., "You put < instead of >", "It should be True, not False.")]

User Feedback (Level 2 - What is the problem): [A brief reply pointing out where the logic breaks without giving the fix (e.g., "The math doesn't make sense there.", "Check your if statement.")]

User Feedback (Level 3 - Just 'you are wrong'): [A very short, blunt reply simply stating it is wrong (e.g., "This doesn't work.", "Wrong logic.", "Nope.")]

Figure 8: Prompt for generating logic reversal corruptions.

Improver	Judge Preference	Causality Inversion	Crucial Omission	Entity Swap	Operator Reversal	Average	w. Feedback Should Win
Large	w. Feedback	28.5%	28.7%	25.9%	25.8%	27.2%	54.6% (N=205)
	w/o. Feedback	36.9%	37.5%	38.7%	37.9%	37.8%	
	Tie	34.6%	33.8%	35.4%	36.3%	35.0%	
Small	w. Feedback	45.7%	50.0%	52.4%	56.4%	51.2%	80.8% (N=574)
	w/o. Feedback	39.2%	37.4%	30.0%	31.2%	34.3%	
	Tie	15.1%	12.6%	17.7%	12.5%	14.5%	

Table 11: Judge pairwise evaluation of the with-feedback vs. without-Feedback variants for the synthetic data.

Feedback Extraction Prompt

You are an expert dialogue analyst. Your task is to review the **entire dialogue** provided below and extract only the User utterances that contain specific feedback regarding the Assistant's performance.

Analysis Instructions:

1. Iterate through every User utterance in the dialogue.
2. Determine if the utterance falls into one of the **Feedback Patterns** (UR1 - UR5) defined below.
3. If the utterance represents **Normal Conversation** (e.g., a new question, a direct answer to a question, or neutral chat), **IGNORE it**. Do not include it in the output.
4. Output a JSON Array containing only the identified feedback instances. If no feedback is found in the entire dialogue, output an empty array [].

Feedback Pattern Definitions:

- **Repeat or Rephrase (UR1):** The user simply repeats or rephrases their previous request because the assistant failed to address it (e.g., "Actually, I wanted...").
- **Make Aware with Correction (UR2):** The user explicitly points out an error AND provides the correct information (e.g., "No, I said Tuesday, not Thursday").
- **Make Aware without Correction (UR3):** The user indicates something is wrong or expresses dissatisfaction but does not provide the fix (e.g., "That is incorrect," "You are wrong").
- **Ask for Clarification (UR4):** The user expresses doubt or asks the assistant to verify its output (e.g., "Are you sure?").
- **Positive Feedback (UR5):** The user explicitly confirms the assistant did a good job or thanks it (e.g., "Great, thanks," "That worked").

Input Data:

[Insert Full Dialogue Here]

Output Format:

Provide **only** a valid JSON list. Do not include markdown formatting.

```
[
  {
    "User Response Pattern": "UR1",
    "User Response Text": "Text of the specific user response"
  },
  {
    "User Response Pattern": "UR5",
    "User Response Text": "Text of another user response"
  }
]
```

Figure 9: Prompt used to extract and categorize user feedback utterances from full conversational dialogues.

Improve with Human Feedback Prompt

Task: Revise the model's response to better address the user's intent based on specific human feedback.

Constraints:

1. **Contextual Seamlessness:** The improved response must be self-contained and blend perfectly into the existing conversation. It must not acknowledge it is a revision or reference the "original" response.
2. **Temporal Integrity:** Use ONLY information that was available at the time of the original user's question. Do not use hindsight or external information not present in the context.
3. **Encapsulation:** The final polished response MUST be wrapped in double brackets [[like this]].

Input Data:

[[CONVERSATION HISTORY]]

{conversation_history}

[[ORIGINAL RESPONSE]]

{original_response}

[[HUMAN FEEDBACK]]

{human_feedback}

Required Output Format:

1. Analysis: Identify how the original response failed to meet the criteria or the human feedback.
2. Comparison: Explain why the improved version is a more seamless and helpful fit.
3. Improved Response: [[<Insert polished response here>]]

Figure 10: Prompt used to revise a model response when human feedback is provided.

Improvement without Feedback Prompt

Task: Revise the model's response to better address the user's intent within the conversation context.

Constraints:

1. **Contextual Seamlessness:** The improved response must be self-contained and blend perfectly into the existing conversation. It must not acknowledge it is a revision or reference the "original" response.
2. **Temporal Integrity:** Use ONLY information that was available at the time of the original user's question. Do not use hindsight or external information not present in the context.
3. **Encapsulation:** The final polished response MUST be wrapped in double brackets [[like this]].

Input Data:

[[CONVERSATION HISTORY]]

{conversation_history}

[[ORIGINAL RESPONSE]]

{original_response}

Required Output Format:

1. Analysis: Identify specific gaps, inaccuracies, or tone shifts in the original response.
2. Comparison: Explain why the improved version is a more seamless and helpful fit.
3. Improved Response: [[<Insert polished response here>]]

Figure 11: Prompt used to revise a model response based purely on the conversation history, without human feedback.

Pairwise Evaluation Prompt

You are an expert judge evaluating AI assistant responses.

Below is a conversation history followed by two candidate responses (Response A and Response B) to the final user message.

Conversation History:

{conversation_history}

Response A:

{response_a}

Response B:

{response_b}

Task:

Evaluate which response better addresses the user's needs in the context of the conversation. Consider:

- Accuracy and factual correctness
- Helpfulness and relevance to the user's question
- Clarity, coherence, and completeness
- Appropriate tone and style for the conversation

Output your verdict as a JSON object and nothing else:

```
{  
  "reasoning": "<one or two sentences explaining your choice>",  
  "verdict": "A" | "B" | "tie"  
}
```

Figure 12: Prompt used by the LLM-as-a-judge to evaluate and compare two candidate model responses.

Issue Resolution Prompt

You are an automated Quality Assurance system evaluating whether a revised AI response successfully incorporates specific user feedback.

Conversation History (including the original response):

{conversation_history}

User Feedback:

{feedback}

Revised Response:

{revised_response}

Task:

The user provided feedback after seeing the original response above. Evaluate whether the revised response adequately addresses that feedback. Consider:

- **Intent matching:** Does the revised response specifically tackle what the feedback asked for?
- **Completeness:** Is the feedback fully addressed, or only partially?
- **Regression:** Did the revision introduce errors or hallucinate content not in the original (unless the feedback requested new content)?

Scoring rubric:

- **1 (Fail):** Feedback completely ignored or the revision is irrelevant to it.
- **2 (Poor):** Attempted to address feedback but failed significantly (wrong direction, factual error).
- **3 (Partial):** Addressed some of the feedback but missed key parts.
- **4 (Good):** Addressed feedback well, with minor gaps.
- **5 (Perfect):** Fully and correctly implemented the feedback without degrading quality.

Output your verdict as a JSON object and nothing else:

```
{
  "feedback_intent": "<one sentence summarizing what the user wanted changed>",
  "change_analysis": "<step-by-step comparison of original vs. revised response with respect to the feedback>",
  "regression_check": "<did the revision break anything else? Yes/No + brief explanation>",
  "score": <1-5>,
  "addresses_feedback": <true or false>
}
```

Figure 13: Prompt used by the issue resolution evaluator to score how well revised responses address the user feedback.

Feedback Relevance Prompt

System Role: You are an AI evaluator. Your task is to analyze user feedback on an AI response to determine if that feedback contains actionable insights that should apply to all future users asking the same question.

The Data:

User Question and AI response:

{conversation}

User Feedback: {feedback}

Is the feedback relevant to improve the response to another user asking the same question?

Definitions:

[[relevant]]: The feedback identifies factual errors, logical flaws, safety issues, or missing crucial information. Correcting this would improve the answer for anyone.

[[irrelevant]]: The feedback is purely subjective (e.g., "I don't like your tone," "Make it shorter") or specific to that user's unique context, and applying it might hurt the experience for a general user.

Instructions:

Analyze the feedback in relation to the question and response.

Determine if the feedback is objective/universal or subjective/personal.

Provide a brief explanation of your reasoning.

End with the final tag:

Final Label: [[tag]]

Figure 14: Prompt used to determine if user feedback contains universally actionable insights.

Prompt 11: Improvement Type Evaluation

You are an expert evaluator analyzing the differences between an original AI response and an improved version.

Conversation History:

{conversation_history}

Original Response:

{original_response}

Improved Response:

{improved_response}

Task:

Compare the original and improved responses. Identify the primary type of improvement made (if any).

Categories:

- **Content: Factuality** — The improved response corrects factual errors or inaccuracies.
- **Content: Completeness** — The improved response adds missing information or addresses parts of the query the original missed.
- **Content: Logic** — The improved response fixes logical errors, inconsistencies, or poor reasoning.
- **Style: Tone** — The improved response uses more appropriate tone or language for the context.
- **Style: Conciseness** — The improved response is more concise without losing important information.
- **Style: Formatting and Structure** — The improved response is better organized, uses better formatting, headers, or structure.
- **No Improvement** — The responses are essentially equivalent, or the "improved" version is not actually better.

Output your verdict as a JSON object and nothing else:

```
{
  "improvement_category": "Select one: [Content: Factuality, Content: Completeness, Content: Logic, Style: Tone, Style: Conciseness, Style: Formatting and Structure, No Improvement]",
  "rationale": "<brief description of the specific gap in the original and how the improved version addresses it>"
}
```

Figure 15: Prompt used to categorize the specific type of improvement made when revising a model response.

ANNOTATOR GUIDELINES: CORRUPTION VERIFICATION

You will be reviewing AI-generated responses that have been deliberately corrupted with a specific error. Your job is to verify whether the corruption actually makes the response wrong or misleading — i.e., whether the corruption "succeeded."

WHAT YOU WILL SEE PER ITEM

Each item presents four fields:

- **User Question:** The original question the AI was asked to answer.
- **Original Response:** The correct, unmodified AI response.
- **Corrupted Response:** The response after a deliberate error was introduced.
- **Edit Log:** A short description of exactly what was changed and why it is an error.

THE FOUR CORRUPTION TYPES

Each item belongs to exactly one corruption type (visible in the filename). Understanding the type helps you know what to look for, but there is no need to verify the corruption type.

1. **Causality Inversion:** A cause-and-effect relationship was reversed. (*Example:* "She failed the exam, which is why she didn't study.")
2. **Crucial Omission:** A critical detail was removed. (*Example:* Removing a required import statement from a code snippet.)
3. **Entity / Subject Swap:** A key entity was replaced with a related but incorrect one. (*Example:* A historical figure replaced with a plausible but wrong person.)
4. **Logic Operator Reversal:** A logical relationship was inverted. (*Example:* A loop condition $i < n$ changed to $i > n$.)

YOUR ANNOTATION TASK

For each item, make one judgment: **Is the corrupted response wrong?**

Ask yourself: *If a user received the corrupted response (without seeing the original), would it give them incorrect information or lead them to a wrong outcome?*

- **Yes:** The corrupted response contains a clear error that would mislead or fail the user.
- **No:** The corrupted response is still acceptable — the change is too minor, or the error doesn't actually affect the outcome.
- **Borderline:** There is a real error, but whether it is "clearly wrong" is genuinely debatable.

Note on Factual Verification:

If the corruption affects factual accuracy in a domain you are unfamiliar with, the edit log alone is sufficient if it convinces you. If it doesn't, you may look up the claim using external resources. You may use an LLM to help locate a relevant source, but not to judge correctness directly. If you still cannot verify it, label "**Unfamiliar**".

SPECIAL NOTES FOR CREATIVE WRITING ITEMS

- Check that the corrupted response still fulfills the user question's requirements.
- Focus on internal consistency: does the corrupted version contradict itself or reverse a story's logic?
- A causality inversion in a motivational story is a real error if the story's message is broken.
- Omitting a character trait or detail that the rest of the story depends on counts as a valid corruption ('**Yes**' label).

Figure 16: Guidelines provided to human annotators for verifying the validity and severity of the generated corruptions.

ANNOTATOR GUIDELINES: RELEVANCE VERIFICATION

You will be reviewing a user's question, an AI's original response, a user feedback that critiques that response, and an AI evaluator's assessment of whether that feedback is universally applicable. Your job is to evaluate whether the AI evaluator correctly tagged the feedback as relevant or irrelevant based on its reasoning and the provided context.

WHAT YOU WILL SEE PER ITEM

- **Question:** The original prompt provided by the user.
- **Model Response:** The AI's original answer to the user's question.
- **Feedback Text (User Response):** The specific critique, correction, or request the user provided regarding the AI's answer.
- **Model Tag:** The AI evaluator's classification of the feedback—either `[[relevant]]` or `[[irrelevant]]`.
- **Model Reasoning:** The explanation for why the AI evaluator chose that specific tag.

YOUR ANNOTATION TASK

For each item, make one judgment: Compare the user's feedback against the AI evaluator's tag and reasoning to determine whether the `model_tag` is correct.

LABELS

Your answer must be a strict **Correct** or **Incorrect**. Use the following definitions to guide your choice:

- **CORRECT** – Select CORRECT if the AI evaluator's tag perfectly matches the true nature of the feedback, supported by sound reasoning.
 - **Accurate Universal Tagging:** The feedback points out a factual error, logical flaw, safety issue, or missing crucial information, and the model correctly tagged it as `[[relevant]]`. Correcting this would improve the answer for anyone asking the same question.
 - **Accurate Subjective Tagging:** The feedback is based on personal preference (e.g., "Make it shorter," "I don't like your tone") or a unique user context, and the model correctly tagged it as `[[irrelevant]]`. Applying this change might hurt a general user's experience.
- **INCORRECT** – Select INCORRECT if the AI evaluator assigned the wrong tag or used fundamentally flawed reasoning. Apply this label if any of the following are true:
 - **Missed Universality:** The user pointed out an objective error (e.g., incorrect math, dangerous advice, broken code), but the model incorrectly tagged it as `[[irrelevant]]`.
 - **False Universality:** The user asked for a purely subjective change (e.g., "Rewrite this as a poem," "I prefer bullet points"), but the model incorrectly tagged it as `[[relevant]]`.
 - **Flawed Reasoning:** The tag happens to be right, but the model's reasoning shows a complete misunderstanding of the feedback or the conversation context.

Figure 17: Guidelines provided to human annotators for verifying the evaluator's classification of user feedback relevance.

ANNOTATOR GUIDELINES: FEEDBACK ADDRESSING

You will be reviewing a conversation of a user with an AI, a user feedback that critiques or corrects the last AI response, and the following AI's revised response. Your job is to evaluate whether the AI's revised response successfully resolves the user feedback.

WHAT YOU WILL SEE PER ITEM

- **Conversation History:** The context of the interaction leading up to the issue.
- **User Feedback:** The specific critique, correction, or request the user provided regarding the previous response.
- **Revised Response:** The new response generated to address the feedback.

YOUR ANNOTATION TASK

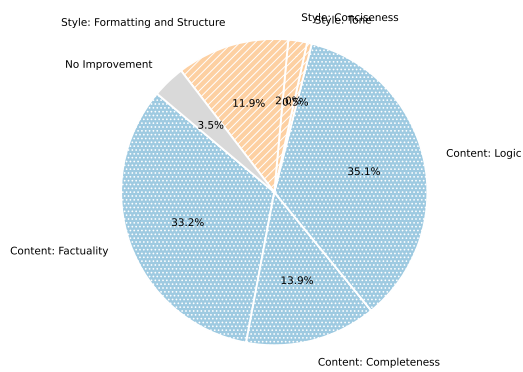
For each item, make one judgment: Compare the revision against the feedback to determine whether the revision addressed the user feedback or not.

LABELS

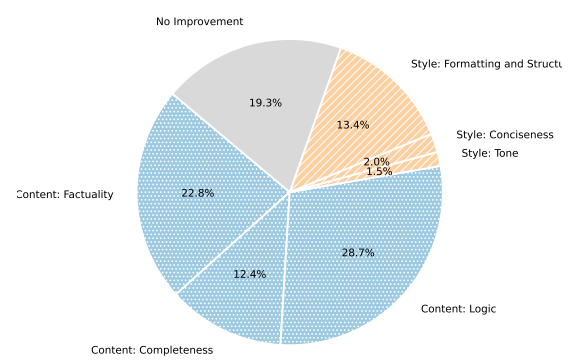
Your answer must be a strict **Yes** or **No**. Use the following definitions to guide your choice:

- **YES** – Select YES if the revised response specifically and successfully tackles the core of what the feedback asked for.
 - **The requirement is met:** If the user asked for a bulleted list, the response is a bulleted list.
 - **The correction is made:** If the user pointed out a factual error, the error is removed or corrected.
 - **Spirit of the prompt:** The revision maintains the original context while seamlessly integrating the requested change.
- **NO** – Select NO if the revised response fails to meet the user's demand. Apply this label if any of the following are true:
 - **Ignored:** The response completely ignores the feedback.
 - **Partial Compliance:** The user asked for two specific things (e.g., "Make it shorter and add a Spanish translation"), but the revision only accomplished one.
 - **Superficial Fix:** The response acknowledges the feedback in text (e.g., "I have updated the tone to be more formal") but the actual content remains unchanged or fails to achieve the requested result.
 - **Regression:** The revision fixes the feedback but severely breaks another critical part of the original prompt/-context (e.g., the tone is fixed, but the answer is suddenly entirely incorrect).

Figure 18: Guidelines provided to human annotators for evaluating whether a revised AI response successfully addressed the user's feedback.



(a) With Feedback



(b) No Feedback

Figure 19: Distribution of improvement types applied to corrupted model responses for the hard prompt category. Content-related improvements (factuality, completeness, and logic) are represented in shades of blue with a dotted texture. Style-related improvements (tone, conciseness, formatting) are represented in shades of orange with a diagonal line texture. Providing feedback significantly increases the proportion of substantive content improvements compared to stylistic edits.

Subset Group	Judge Preference		
	w. Feedback	w/o. Feedback	Tie
Independent Success	87.7%	12.3%	0%
Feedback Dependent	72.2%	27.8%	0%

Table 12: Judgments Distribution: Group A vs. Group B

Baseline	Judge Preference		
	Reimprove w. Feedback	Baseline	Tie
w/o. Feedback	51.0%	37.6%	11.8%
Reimprove w/o. Feedback	28.9%	57.3%	13.7%

Table 13: Reimprovements results. The judge prefers the two stage with-feedback improvement over the single stage baseline, but when comparing against two stage without-feedback its reference reversed.

Improver	Variant	Average
GPT-OSS	w/o feedback	48.5%
	w/ feedback	54.7%
		$\Delta + 6.2\%$

Table 14: Percentage of improved responses successfully addressing the issues on the hard prompt category with GPT-OSS as the improver. We observe a consistent positive delta between the without-feedback and with-feedback variants.

Improver	Judge Preference	Average	WFSouldWin
GPT-OSS	w. Feedback	42.9%	95.2%
	w/o. Feedback	40.6%	4.8%
	Tie	16.5%	0.0%

Table 15: Judge pairwise evaluation of the with-feedback vs. without-feedback variants for the openai_gpt-oss-20b improver, averaged across corruption types. The ‘WFSouldWin‘ column represents the subset of cases where the with-feedback variant successfully addressed the issue while the without-feedback variant did not.